

基于Cursor + OpenSpec 的开发 workflow

mp.weixin.qq.com/s/ZNgxn8R_Pahn6c_KgJXAw

AIPlayer

一、前言

你有没有遇到过这种场景：让 AI 改个功能，它倒是改了，但改的方向跟你想的完全不一样，甚至把已有逻辑搞坏了？问题不出在 AI 不够聪明，而是你没告诉它"边界在哪"。

二、快是真已，但方向对吗

说到 AI 辅助开发，大多数人的第一反应是"让 Cursor 帮我写"——打开编辑器，描述一下需求，Agent 模式一跑，代码就出来了。快是真快，但坑也是真多。

举个真实的例子。我们有一个 CI 流水线需要统计 API 覆盖率。最初的实现很粗糙：用正则扫描 name(这种调用点来判断"某个 API 被用到了没有"。问题显而易见——宏、函数指针、条件编译，全都会被误判。后来我们想把它升级成基于 AST 的精确统计。这事儿说起来一句话，但真做起来要回答的问题多了去了：

- 要不要引入 libclang？
- CI 镜像会不会变大？
- compile_commands.json怎么拿到？
- 构建系统是 CMake 还是别的？
- GCC 编译的项目用 clang 做 AST 解析，flag 兼容性怎么处理？
- 解析失败了怎么办？CI 能不能不阻断？
- 顺带还要加上语句/分支覆盖率——这又是另一条线。

如果直接跟 Cursor 说"把 API 覆盖率升级成 AST 的"，你猜它会怎么干？大概率上来就改代码，改到一半发现方向不对，再推倒重来。改三轮你就烦了。

这就是为什么需要 OpenSpec：在让 AI 动手之前，先用结构化的方式把"要做什么、为什么做、怎么做、边界在哪"写清楚。

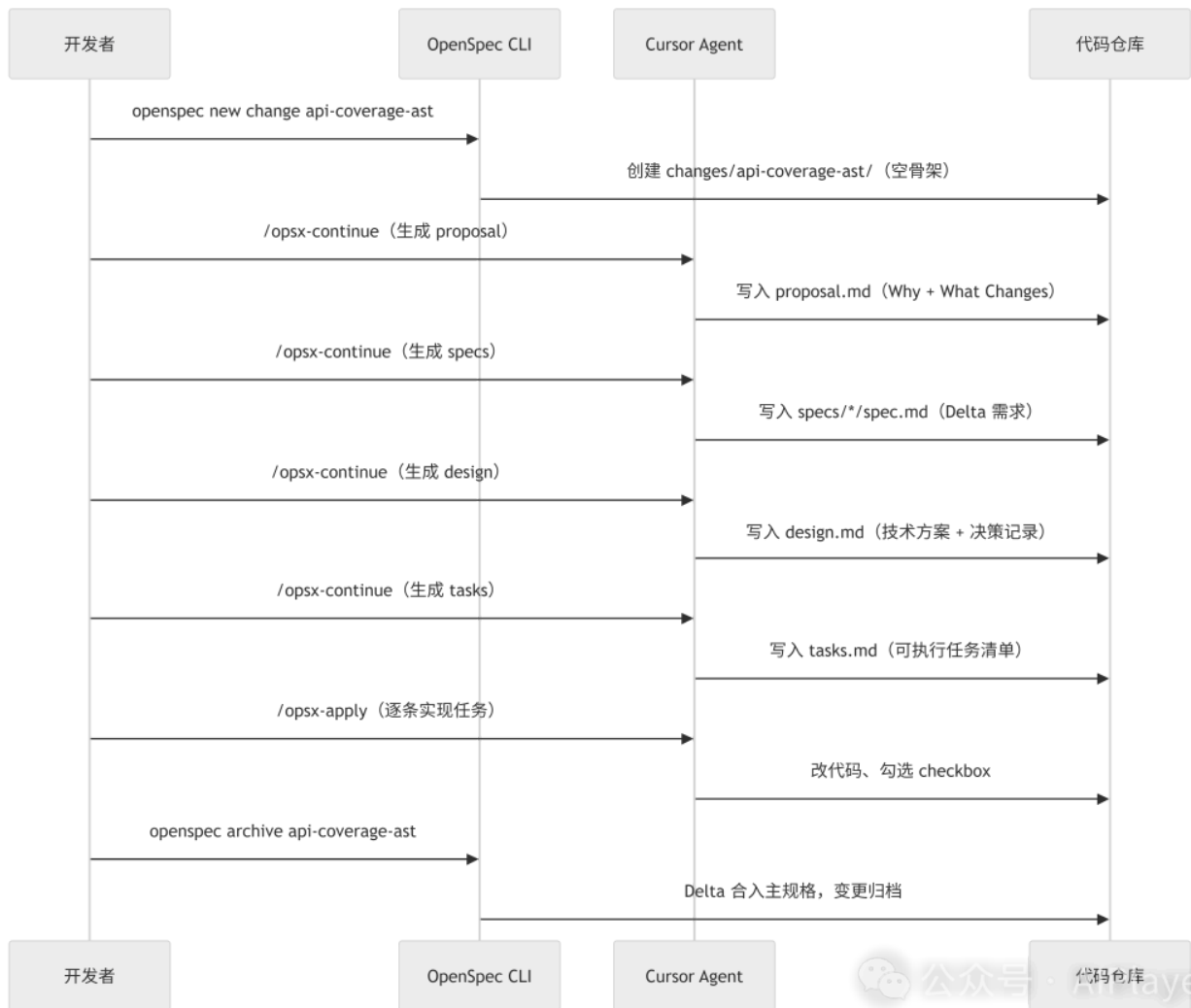
三、OpenSpec 是什么

OpenSpec 是一个开源的 spec-driven 开发工具。它做的事情不复杂：用一套约定好的 Markdown 格式，把一次代码变更拆解成：

proposal → specs → design → tasks 四个阶段，每个阶段生成一份"制品" (artifact)，后一个阶段依赖前一个阶段的产出。

听起来像传统的需求文档流程？区别在于：这些 Markdown 制品不是给人看的备忘录，而是直接喂给 AI Agent 的上下文。Cursor 读完这些文件就知道该做什么、不该碰什么。

回到刚才 API 覆盖率的例子，整个流程走下来是这样的：



每一步，Cursor Agent 都能看到前面的所有制品。比如在生成 `design.md` 时，它已经读过 `proposal.md` 和 `specs/*/spec.md`，知道"要做什么"和"需求的边界"，不会胡编方案。到写代码时，`tasks.md` 里列得清清楚楚每条任务干什么，改哪个文件，验证标准是什么。

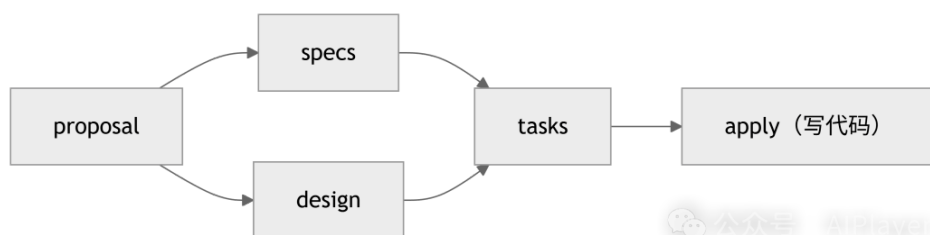
你可能会问：这跟直接在 Cursor 对话里写一堆上下文有什么区别？

区别在于 **可复用、可追溯、可验证**。Markdown 文件提交到 Git，团队成员都能看到决策过程。验收时跑 `openspec verify`，自动检查任务完成情况和规格一致性。归档时 `openspec archive` 把 delta 需求自动合入主规格文件——不用手动同步文档。

四、制品依赖

OpenSpec 最有意思的设计不是"四个 Markdown 模板"，而是它底层的 Artifact DAG（有向无环图）。

在默认的 spec-driven 工作流里，四个制品的依赖关系不是线性的 $A \rightarrow B \rightarrow C \rightarrow D$ ，而是这样的：



proposal 是起点，specs 和 design 可以并行生成（它们只依赖 proposal），而 tasks 必须等这两个都完成才能生成。这意味着你可以先写需求（specs），同时让另一个人做技术方案（design），最后再合并拆任务。但这个 DAG 不是硬编码的。它在 schema.yaml 里声明：

```
name: spec-driven
version: 1
artifacts:
  - id: proposal    generates: proposal.md
    requires: []    - id: specs    generates: "specs/**/*.*md"    requires: [proposal]
  - id: design    generates: design.md    requires: [proposal]
  - id: tasks    generates: tasks.md    requires: [specs, design]
apply: requires: [tasks] tracks: tasks.md
```

requires 字段声明依赖，generates 字段告诉系统“这个制品完成后会产出什么文件”。系统通过文件系统检测哪些制品已完成——看到 proposal.md 存在，proposal 就标记为 done。跑一下 openspec status，输出类似：

```
Change: api-coverage-ast (spec-driven)Progress: 4/4 artifacts complete
  ✓ proposal  ✓ specs  ✓ design  ✓ tasks
```

再看下一步能干什么，系统用 Kahn 拓扑排序 计算出所有依赖已满足的制品。代码核心逻辑不到 30 行：

```
getBuildOrder(): string[] {  const inDegree = new Map<string, number>
  ();  const dependents = new Map<string, string[]>();  for (const artifact
of this.artifacts.values()) {    inDegree.set(artifact.id, artifact.requires.length);
    dependents.set(artifact.id, []);  }  for (const artifact of this.artifacts.values())
{    for (const req of artifact.requires) {
dependents.get(req)!.push(artifact.id);    }  }  const queue =
[...this.artifacts.keys()]    .filter(id => inDegree.get(id)
=== 0).sort();  const result: string[] = [];  while (queue.length > 0) {
    const current = queue.shift()!;    result.push(current);    for (const dep of
dependents.get(current)!) {      const newDegree = inDegree.get(dep)! - 1;
inDegree.set(dep, newDegree);      if (newDegree === 0) queue.push(dep);    }
  }  return result;}
```

你可以自定义 schema，增减制品、改变依赖关系。比如如果团队不需要 design 阶段，去掉就行。

五、Delta Spec 机制

传统做法是直接改需求文档——虽然也有版本记录，但改动散落在各处，时间一长很难把“需求变更”和“代码变更”对应起来。OpenSpec 用了一个巧妙的设计：Delta Spec。

每个 change 不直接修改主规格文件（openspec/specs/xxx/spec.md），而是在自己的目录下创建同名的 delta 文件：

changes/my-change/specs/xxx/spec.md

只写“变更的部分”。格式是这样的：

```
## ADDED Requirements
### Requirement: compile_commands.json discovery (MUST)The system MUST locate a valid
compile_commands.jsonfor the api_test build context.
##### Scenario: Compile DB present- **WHEN** api_test 任务结束后开始覆盖率统计- **THEN** 系统
MUST 在预定义位置查找 compile_commands.json
## MODIFIED Requirements
### Requirement: Covered API identification (MUST)- **WHEN** 系统从 api_test_apps 统计
called_apis- **THEN** 系统 MUST 以 AST 识别真实函数调用
```

四种操作：ADDED、MODIFIED、REMOVED、RENAMED。

归档时，系统按 RENAMED → REMOVED → MODIFIED → ADDED 的确定性顺序把这些变更合入主规格——先改名、再删除、再修改、最后新增，避免冲突。

合并前还会做冲突检测：比如同一个需求不能既出现在 MODIFIED 又出现在 REMOVED 里。这套机制保证了需求文档的演进是可追踪的、可审计的。不用翻 Git log 找“这个需求到底什么时候改的、为什么改”——每个 change 目录就是一次完整的变更记录。

六、怎么对接Cursor

OpenSpec 支持 22 种 AI 工具的适配器（Cursor、Claude、GitHub Copilot、Windsurf 等），但核心原理都一样：把结构化指令写成工具能识别的格式文件。

以 Cursor 为例，适配器做的事情就是生成 .cursor/commands/opsx-*.md 文件：

```
export const cursorAdapter: ToolCommandAdapter =
{  toolId: 'cursor',  getFilePath(commandId: string): string {
    return path.join('.cursor', 'commands', `opsx-${commandId}.md`);
  },  formatFile(content: CommandContent): string {    return `---name:
/opsx-${content.id}id:
opsx-${content.id}category: ${content.category}description: ${content.description}---
${content.body}`;  },};
```

生成后的文件长什么样呢？

拿 /opsx-continue 命令举例——它是平时用得最多的一个，功能是“继续推进当前 change，生成下一个制品”。Cursor 里输入 /opsx-continue，Agent 就会：

- 找到当前仓库的 openspec/ 目录
- 运行 `npx openspec status --change <name>` 查看哪些制品已完成
- 运行 `npx openspec instructions --change <name> --artifact <next>` 拿到指令
- 按指令里的模板、依赖、规则，生成对应的 Markdown 文件

指令输出是 XML 格式的结构化 prompt，包含 <task>、<dependencies>、<template>、<rules>、<project_context> 等标签。AI Agent 读到的不是一句模糊的“帮我写个设计文档”，而是：

- 你要生成的是 design.md
- 它依赖 proposal（已完成）和 specs（已完成），内容在 changes/api-coverage-ast/ 目录
- 模板结构是：Context、Goals / Non-Goals、Decisions、Risks / Trade-offs
- 项目约定是：TypeScript 风格、conventional commits

这就是为什么 Cursor 在这种模式下“不容易跑偏”，它拿到的上下文足够具体，边界足够清晰。

七、实战演练

来看我们团队的真实案例：把 API 覆盖率从正则扫描升级成 AST 分析。

第一步：创建变更

```
npx openspec new change api-coverage-ast
```

生成 openspec/changes/api-coverage-ast/ 目录，里面只有空的制品模板。

第二步：写 Proposal

在 Cursor 里执行 /opsx-continue，Agent 根据我的口头描述，生成了这份 proposal：

Why: 当前 api-coverage-report 采用静态扫描 (prototype + name(调用点) 作为"接口触达"指标, 存在误计/漏计风险 (宏、函数指针、间接调用、条件编译等)。What Changes: 将覆盖统计方式从 static_call_scan 升级为 AST/compile_commands.json 驱动; 继续保持统计范围仅限 5 个 include 目录; 提供降级策略; 顺带补充语句/分支覆盖率。

注意这里 Agent 的输出不是随便写的——它遵循了 OpenSpec 模板的 ## Why + ## What Changes + ## Capabilities + ## Impact 结构。我审核了一下, 调了几句话, 就提交了。

第三步: 生成 Delta Specs

再跑一次 /opsx-continue。这次 Agent 读完 proposal 后, 自动识别出需要修改两个能力:

- api-coverage-report: 已有能力, 需 MODIFIED + ADDED
- code-coverage-report: 新能力, 纯 ADDED

它在 changes/api-coverage-ast/specs/ 下生成了两个 delta spec 文件。拿 api-coverage-report 的 delta 来说, Agent 加了三个新需求 (compile DB 发现、GCC 兼容、降级策略) 并修改了两个已有需求 (覆盖率统计方法和 JSON 输出格式)。每个需求都有 WHEN/THEN 格式的验收场景。

这些写得够不够好? 说实话, 初版不完美。但因为它格式清晰, review 很快——我调整了几个 MUST/MAY 的粒度就行了。比起从零写需求文档, 效率提高太多了。

第四步: Design 和 Tasks

Design 阶段 Agent 给出了两个实现方案 (Python + libclang vs 独立 clang-tool), 列出了各自的优缺点。我选了方案 A, Agent 接着生成 tasks:

```
## 1. Decide compile DB source and AST implementation option- [x] 1.1 确认
compile_commands.json 的生成与落点- [x] 1.2 确定 AST 解析实现落地方式: 选择 OptionA
(Python+libclang) - [x] 1.3 降级策略确定为: 回退到 static_call_scan## 2. Implement AST-
based called API extraction- [x] 2.1 新增 AST 扫描实现- [x] 2.2 确保宏不被当作函数调用计入-
[x] 2.3 仅统计主规格规定的 exported APIs## 5. Verify and archive- [ ] 5.1 在 CI 容器内验证
AST 统计可跑通- [ ] 5.2 验证邮件展示- [ ] 5.3 openspec verify 通过后归档变更
```

第五步: Apply

执行 /opsx-apply。Cursor Agent 会逐条读取未完成任务, 结合 design 和 specs 里的约束, 开始改代码。完成一条就在 tasks.md 里勾选对应的 checkbox。

这个过程中最有价值的是: Agent 不会乱改。它知道"降级策略是回退到 static_call_scan", 就不会在解析失败时让 CI 挂掉。这些边界全在 specs 和 design 里写着。

第六步: 归档

```
npx openspec archive api-coverage-ast
```

系统自动把 delta specs 合入主规格文件:

```
openspec/specs/api-coverage-report/spec.md
```

把整个 change 目录移到 changes/archive/2026-01-26-api-coverage-ast/。主规格现在包含了 AST 相关的所有新增和修改后的需求, 不需要手动 copy-paste。

One More Thing:

如果你觉得以上的步骤太多了, 更喜欢直接简单的流程, 那下面的方式更适合你:

直接告诉 AI 让它基于 openspec 工作流程来进行工作, 给到的 prompt 为

使用 openspec 工作流 (工作流的每个阶段都要经过我的确认, 当所有阶段都完成后让我确认是否可以归档) 完成以下需求: 针对 api_test 的 job, 增加一个统计覆盖率的命令选项, 当使能时, 只编译并统计覆盖率, 不需要跑用例, 未使能时, 跑用例并统计覆盖率。

这样Cursor就会自己执行每个阶段的相关命令，并且会在每个阶段完成后要求你对产物进行review，问你是否通过，如果你觉得不通过，就提出修改点，它就会继续修改直到你满意再进入下一阶段，整个过程不需要你执行命令，更直接高效。

八、三种流程对比

维度	纯 Cursor Agent	传统需求文档	Cursor + OpenSpec
需求表达	对话框里口头描述，随写随忘	详细但冗长，不被代码工具消费	结构化 Markdown，AI 和人都能读
AI 上下文质量	只有当前对话窗口	不参与编码过程	完整的 proposal → specs → design → tasks 链
变更追溯	无，对话历史丢失	有，但靠人肉维护版本	Git 版本控制 + Delta Spec 自动合并
边界控制	弱，Agent 容易跑偏	有文档但 Agent 看不到	强，specs 里写的就是 Agent 的约束
验收标准	没有，凭感觉看看	有但人工比对	WHEN/THEN 场景 + openspec verify
团队协作	单人单会话	协作成本高（文档评审）	标准化制品 + Git Review
上手成本	几乎为零	低（但维护成本高）	中（需要理解制品关系）
适合场景	小改动、原型验证	正式项目交付文档	中大型功能开发、跨模块变更

什么时候不需要 OpenSpec？改个 bug、加行日志、调整 UI 细节——这些直接用 Cursor 就行了，拉一套 spec 流程纯属浪费时间。

什么时候值得用？像我们这个 API 覆盖率升级——涉及多个模块、有技术选型要做、需要降级策略、还要保持 CI 兼容性——这种场景用 OpenSpec 可以明显减少 AI "改跑偏"的次数。

九、总结

第一，AI 写代码的瓶颈不是"写"，是"上下文"。 Cursor Agent 的能力上限取决于你喂给它多少、多准确的上下文。OpenSpec 做的就是吧"想清楚"这个步骤结构化，让 AI 拿到的信息不是模糊的口头描述，而是有依赖关系、有验收标准的制品链。

第二，Spec-Driven 不是回到瀑布模型。 OpenSpec 的制品是轻量的 Markdown，不是几十页的需求规格说明书。每个制品几百字到一两千字，写起来几分钟。而且 AI 帮你写初稿，你只需要 review 和微调。开发速度不是变慢了，是把"反复修正 AI 输出"的时间前移到了"一次性把需求说清楚"。

第三，不是所有项目都需要它。 个人小项目、快速原型、简单 bug fix——直接用 Cursor 就好。但如果你的团队在做中大型功能开发，尤其是涉及架构决策、多模块联动、需要可追溯的需求管理——试试 OpenSpec，它可能会改变你对 AI 辅助开发的认知。项目地址：
<https://github.com/Fission-AI/OpenSpec>